

Learning the globe?

Kirke Joamets Kaarel Hänni others?

May 2024

1 Notation

- Let m be the number of locations (cities).
- Let n be the dimension of the embedding space.

2 Research plan

2.1 Training and analyzing a simplest model

For a very simple case, we could pick a single (small) country, and see if we learn its 2D geometry. We could even make the embedding space be 2-dimensional — this lets us

1. Find a data set of pairwise distances between pairs of cities in a (small) country. In this case, it doesn't matter if you find the geodesic distances.¹
2. The model's parameters are just a $2 \times m$ matrix W .
3. The cities get some arbitrary ordering (e.g., alphabetical). Each city gets assigned a one-hot vector in $\mathbb{R}^m$ accordingly.
4. On inputs i and j with one-hot vectors $\mathbf{e}_i$ and $\mathbf{e}_j$, the model outputs $d(\mathbf{e}_i, \mathbf{e}_j)^2 = \|W\mathbf{e}_i - W\mathbf{e}_j\|^2 = \|W(e_i - e_j)\|^2$.
5. Let's use MSE loss on the squared distance. That is, letting $d(i, j)$ denote the actual distance on input (i, j) , the loss on that data point would be $\left(\|W(e_i - e_j)\|^2 - d(i, j)^2\right)^2$.
6. Then do (batch?) gradient descent to find a W which gets low loss.

¹Later, it will matter whether you find the geodesic or 3D-straight-line distances, but for a small patch of the globe, these will be almost equal anyway. You might want to get both geodesic and 3D-straight-line distance data sets for a large set of locations on Earth for later experiments though. If these are not readily available, you can write code to compute these from coordinates.

7. Make an animation showing how the points (the columns of W) move around during training.
8. Does it find a global optimum, or does it get stuck?

2.2 More stuff to do

1. Make n , the embedding dimension, larger. In your final animation, you then probably want to only include the 2 or 3 top principal components of the data set at each time. There might be some trickiness in getting the components to line up across different time steps? Another option would be to just always plot the projection to the final top PCA axes.
2. Use the entire globe, not just a small patch.
3. By default, you'd do this with 3D distances, but it might also be interesting to try it with geodesic distances.
4. Maybe analyze the convergence to a 2/3-dimensional subspace quantitatively. More generally, see if you notice some interesting patterns.
5. Train on only a subset of the data, and check that it still learns the correct geometry. How small can the data set be? I'm not sure what the theoretically correct answer is here — it's not that the min cut is at least 3. How close to hitting the theoretical bound do we get empirically?
 - (a) Take a range of densities p from 0 to 1. Maybe take the densities to be spaced uniformly in log-space from $1/m^2$ to 1.
 - (b) For each p , generate a random graph with density p , and check if each vertex in it has ≥ 3 neighbors. More precisely, this should be the right thing for the case where the embedding space dimension is 2. For the case where we're doing the entire globe and the embedding space dimension is 3, I think we should check for every vtx having ≥ 4 neighbors. If the embedding space dimension is larger, I'm not sure what the right condition to check is.
 - (c) Edges in this graph correspond in distances to include in the train data set.
 - (d) For each random graph, train the NN with the corresponding train data set, and record the final test loss as a function of p .
 - (e) Plot loss and also whether each vtx had ≥ 3 neighbors (or some alternative condition if the dim is not 2) as a function of p .
 - (f) Does loss go down to pretty much 0 precisely around when it is first the case that every city has ≥ 3 neighbors? (I.e., do all cities start to be pretty much correctly placed then?)
6. (maybe less interesting: Add a regularization term, see how it affects the process / the final learned configuration.)

7. Train SAEs on embeddings and figure out what they are finding — in particular, whether it is just clustering (perhaps you will find that SAE directions just correspond to city clusters or whatever).
8. Do everything above with a transformer model. Maybe one just with attention initially? See if it can still solve the task well, try to figure out how it's solving it, and try to explain what happens during training. Maybe one can see grokking here: if you make the data set only a small fraction of all pairs of cities, will you initially memorize these pairs, but after a while (with weight regularization) learn a solution that also works on pairs that you didn't see in training? Perhaps we can't really understand what's going on internally, but we can empirically see that we get as close to the theoretical bound of learning the structure from the subset of distances as for the other architecture? (open-ended)

3 How to present the findings

1. One story we could tell with this is Dmitry's story of many memorizations forcing a generalization: here, the memorizations are the individual edges forcing the right distances also for the other edges. Maybe we could even publish this together with Dmitry's and Nina's results on a similar thing in modular arithmetic?